

Manufacturing Operational Insights Report

Deploy Manufacturing App to Kubernetes

Course Name: Data Analytics

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	AADITI BHALE	EN22CS303001
2.	ABHIGYAN SINGH THAKUR	EN22CS303004
3.	ADITYA PUROHIT	EN22CS303007
4.	DHRUV JAIN	EN22CS304022
5.	DIVYANSH TYAGI	EN22CS304024

Group Name: 06D11

Project Number:032

Industry Mentor Name: AASHRUTI SHAH

University Mentor Name: SHYAM PATEL

Academic Year:2022-2026

1.Problem Statement & Objectives

- 1.1 Problem Statement
- 1.2 Project Objectives
- 1.3 Scope of the Project

2.Proposed Solution

- 2.1 Key features
- 2.2 FlowChart
- 2.3 Overall Architecture / Workflow
- 2.4 Tools & Technologies Used

3.Results & Output

- 3.1 Docker Build Process
- 3.2 Docker Push to Docker Hub
- 3.3 Docker Hub Repository View
- 3.4 Kubernetes Deployment And Service Creation
- 3.5 Running Pods Verification
- 3.6 NodePort Service Verification
- 3.7 Applicable UI – Verison V1
- 3.8 Applicable Ui – Version v2(Rolling Updates)

4. Reports /Dashboards /Models

- 4.1. System Testing & Validation
- 4.2. Performance & Realiability
- 4.3. Devops Principle Applied
- 4.4. Security Considerations
- 4.5. Comparative Analysis

5. Key Outcomes

6.Practical Industry Relevance

7. Limitations

8.Conclusion

9.Future Scope & Enhancements

1. Problem Statement & Objectives

1.1 Problem Statement

In modern manufacturing environments, applications must be highly available, scalable, and capable of zero-downtime updates. Traditional deployment methods often cause service interruptions during upgrades, lack scalability, and require manual infrastructure management.

The problem addressed in this project is to design and deploy a containerized Manufacturing Application that:

- Ensures zero downtime during version upgrades
- Supports scalability through container orchestration
- Provides monitoring endpoints
- Uses modern DevOps practices

The system must be deployed using Docker and Kubernetes to simulate a real-world production-grade deployment environment.

The rapid evolution of cloud computing and DevOps practices has transformed the way modern applications are developed, deployed, and maintained. Traditional deployment approaches often suffer from issues such as downtime during updates, limited scalability, manual infrastructure management, and environment inconsistency.

Containerization and orchestration technologies such as Docker and Kubernetes have revolutionized application lifecycle management by providing portability, scalability, automation, and fault tolerance.

This project focuses on designing and deploying a containerized Manufacturing Application using Docker and Kubernetes. The objective is to simulate a production-ready environment where applications can be deployed, updated, and managed efficiently with minimal downtime.

The system demonstrates zero-downtime rolling updates, self-healing pods, and service exposure using Kubernetes NodePort.

1.2 Project Objectives

The primary objectives of this project are:

- To containerize a Python Flask Manufacturing application using Docker.
- To push the Docker image to Docker Hub.
- To deploy the application using Kubernetes Deployment.
- To expose the application using a NodePort Service.
- To implement zero-downtime rolling updates.
- To validate application health using health endpoints.
- To demonstrate pod self-healing behavior.
- To simulate production-grade deployment practices.

1.3 Scope of the Project

This project focuses on:

- Application containerization
- Kubernetes orchestration
- Rolling update implementation
- Basic health monitoring
- NodePort exposure

This project does not include:

- CI/CD automation
- Cloud deployment (AWS/Azure/GCP)
- Database integration
- Production security hardening

2. Proposed Solution

In manufacturing industries, monitoring applications must remain available 24/7 to ensure uninterrupted operations. Even minor downtime during updates can lead to production losses and inefficiencies.

Key challenges in traditional deployment:

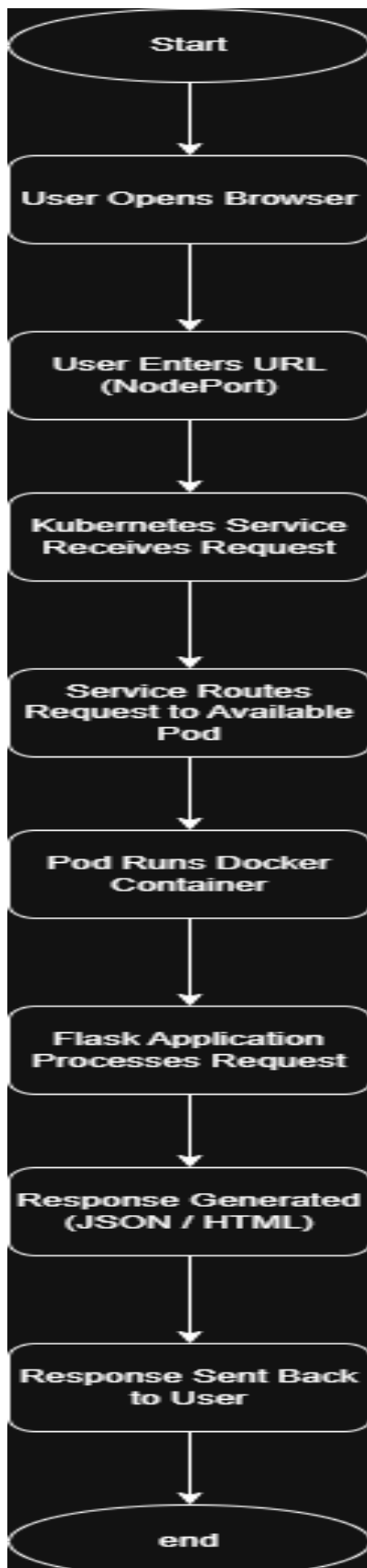
- Manual server configuration
- Application dependency conflicts
- Downtime during version updates
- Poor scalability
- Lack of automated recovery mechanisms
- Inconsistent development and production environments

To overcome these challenges, containerization and orchestration are required.

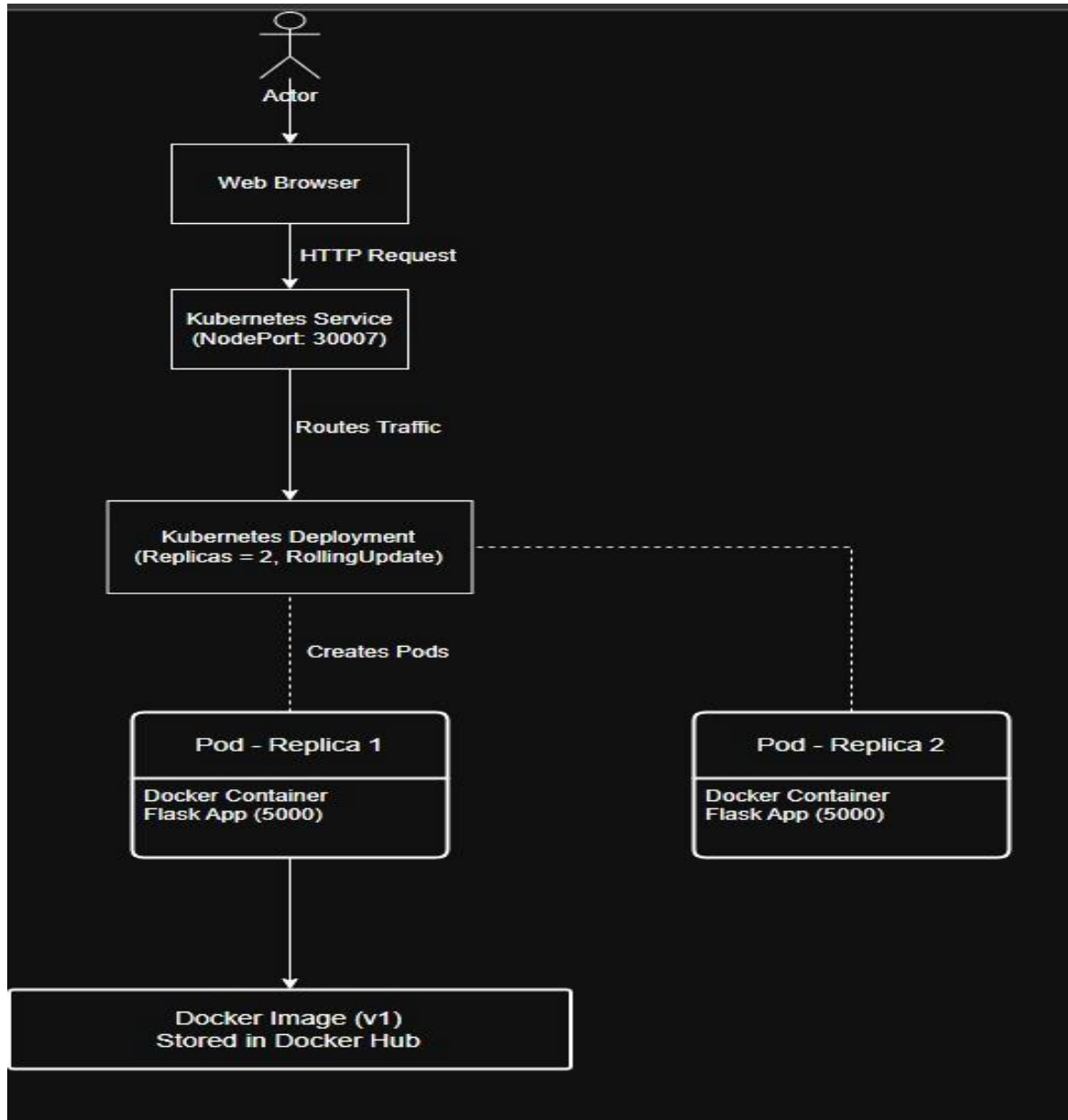
2.1 Key Features

- Docker-based containerization
- Kubernetes Deployment with replicas
- NodePort service exposure
- Health and Status endpoints
- Rolling update support
- Self-healing pods
- Version management (v1 → v2)
- Zero-downtime deployment

2.3 FlowChart :



2.2 Overall Architecture / Workflow



Explanation Content:

The architecture consists of:

User → NodePort Service → Kubernetes Deployment → Pods → Docker Container → Flask Application

Workflow:

1. User accesses the application through NodePort (30007).
2. Service forwards traffic to available Pods.
3. Deployment manages replicas.
4. Pods run Docker container.
5. Container runs Flask app.
6. Rolling update replaces pods gradually.

2.3 Tools & Technologies Used

Tool	Purpose
Python	Application development
Flask	Web framework
Docker	Containerization
Docker Hub	Image registry
Kubernetes	Container orchestration
kubectl	Cluster management
YAML	Deployment configuration
NodePort	External access mechanism

3. Results & Output

The proposed architecture follows a layered container-based deployment model:

Layer 1: User Layer

Users access the application through a web browser using the NodePort.

Layer 2: Kubernetes Service Layer

The NodePort Service exposes the application externally and routes traffic to available pods.

Layer 3: Deployment Layer

Kubernetes Deployment manages replica count and ensures rolling updates.

Layer 4: Pod Layer

Pods act as execution units containing Docker containers.

Layer 5: Container Layer

Docker container encapsulates the application and dependencies.

Layer 6: Application Layer

Flask-based Manufacturing Dashboard application.

This architecture ensures modularity, scalability, and fault tolerance.

3.1 Docker Build Process

```
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> docker build -t aaditibhale/manufacturing-app:v2 .
[+] Building 14.8s (11/11) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.2s
=> => transferring dockerfile: 241B                                0.1s
=> [internal] load metadata for docker.io/library/python:3.10-slim 3.4s
=> [auth] library/python:pull token for registry-1.docker.io       0.0s
=> [internal] load .dockerignore                                  0.1s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/python:3.10-slim@sha256:e508a34e5491225a76fbb9e0f43ebde1f691c6a689d096d7510cf7fb17d4ba6f 0.3s
=> => resolve docker.io/library/python:3.10-slim@sha256:e508a34e5491225a76fbb9e0f43ebde1f691c6a689d096d7510cf7fb17d4ba6f 0.1s
=> [internal] load build context                                   0.2s
=> => transferring context: 2.44kB                                    0.0s
=> [2/5] WORKDIR /app                                              0.2s
=> [3/5] COPY app/requirements.txt .                               0.2s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt        7.8s
=> [5/5] COPY app/ .                                               0.1s
=> exporting to image                                              1.9s
=> => exporting layers                                              1.1s
=> => exporting manifest sha256:85a42de24c2a2f357cb5955950e9e49e80eb7a83175479dfa8e7f499022c66f5 0.0s
=> => exporting config sha256:0995e5eea437fc6b6ad4816f776f089b8e319dcc410d494a70cc02d96214e468 0.0s
=> => exporting attestation manifest sha256:a387858e3a5e20f22b12183733b7fe67f8503edbd19cfca3ca33ed93552ed6b5 0.1s
=> => exporting manifest list sha256:46dd65f2fb7350225f0ed9ce713c31cc1110e4e33ac8b41d8e943cd2a456e129 0.0s
=> => naming to docker.io/aaditibhale/manufacturing-app:v2        0.0s
=> => unpacking to docker.io/aaditibhale/manufacturing-app:v2     0.5s
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> docker push aaditibhale/manufacturing-app:v2
The push refers to repository [docker.io/aaditibhale/manufacturing-app]
619a06b1137: Pushed
d0d1977db485: Layer already exists
0c8d55a45c0d: Layer already exists
8a32d079eb94: Pushed
1fe590ebe5c3: Pushed
63ebb7511fc1: Layer already exists
626baa6cf1aa: Pushed
```

The Docker image was built using a multi-layer Dockerfile based on Python 3.10 slim image. The application dependencies were installed via requirements.txt and the application source code was copied into the container.

The image was successfully built and tagged as:

aaditibhale/manufacturing-app:v2

This confirms successful containerization of the application.

3.2 Docker Push to Docker Hub

```
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> docker push aaditbhale/manufacturing-app:v2
The push refers to repository [docker.io/aaditbhale/manufacturing-app]
619a06ba1137: Pushed
d0d1977db485: Layer already exists
0c8d55a45c0d: Layer already exists
8a32d079eb94: Pushed
1fe590ebe5c3: Pushed
63ebb7511fc1: Layer already exists
626baa6cf1aa: Pushed
690eaffcf0e9: Layer already exists
cff1a20f4ab4: Pushed
v2: digest: sha256:46dd65f2fb7350225f0ed9ce713c31cc1110e4e33ac8b41d8e943cd2a456e129 size: 856
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl apply -f deployment.yaml
deployment.apps/manufacturing-app configured
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> |
```

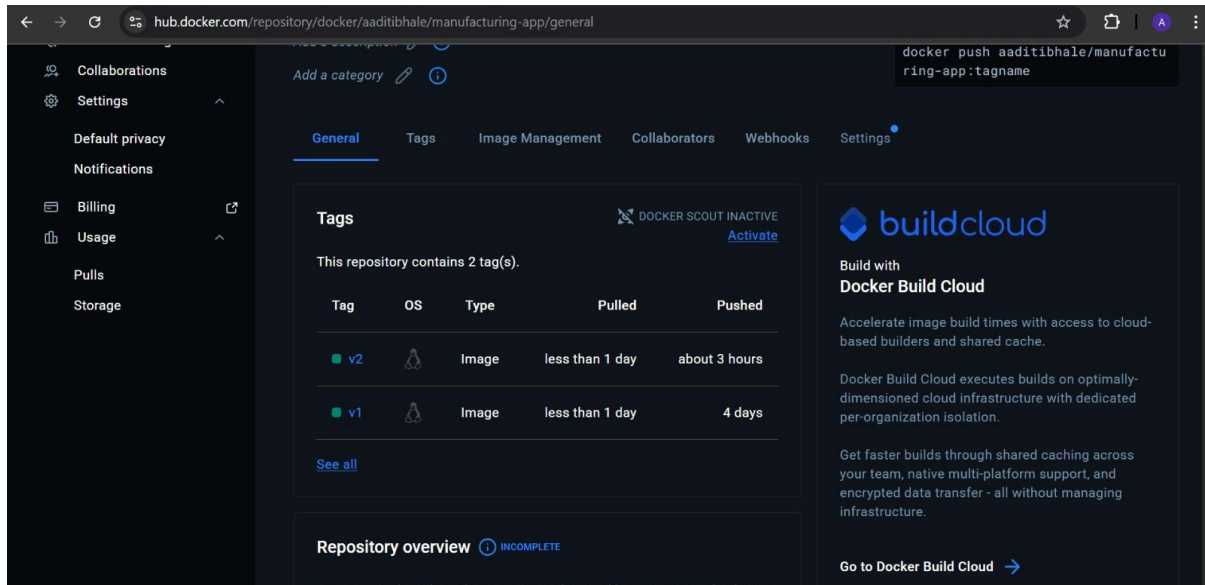
The Docker image was successfully pushed to Docker Hub. The push operation uploaded only the modified layers, demonstrating Docker’s layered architecture efficiency.

Two versions were maintained:

- v1
- v2

This enables version control and rollback capability.

3.3 Docker Hub Repository View



The Docker Hub repository confirms that two versions of the image are available:

- Version v1
- Version v2

This ensures version traceability and supports rolling update deployment in Kubernetes.

3.4 Kubernetes Deployment & Service Creation

```
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> docker push aaditibhale/manufacturing-app:v2
The push refers to repository [docker.io/aaditibhale/manufacturing-app]
619a06ba1137: Pushed
d0d1977db485: Layer already exists
0c8d55a45c0d: Layer already exists
8a32d079eb94: Pushed
1fe590ebe5c3: Pushed
63ebb7511fc1: Layer already exists
626baa6cf1aa: Pushed
690eaffcf0e9: Layer already exists
cff1a20f4ab4: Pushed
v2: digest: sha256:46dd65f2fb7350225f0ed9ce713c31cc1110e4e33ac8b41d8e943cd2a456e129 size: 856
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl apply -f deployment.yaml
deployment.apps/manufacturing-app configured
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> |
```

The deployment.yaml file created a Kubernetes Deployment with 2 replicas. The service.yaml file created a NodePort service exposing the application externally.

Deployment ensures:

- High availability
- Replica management
- Rolling update support

3.5 Running Pods Verification

```
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
manufacturing-app-7f498dd7cd-7l54z 1/1     Running   0           3h7m
manufacturing-app-7f498dd7cd-8h7tz 1/1     Running   0           33s
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
manufacturing-app-7f498dd7cd-7l54z 1/1     Running   0           3h8m
manufacturing-app-7f498dd7cd-8h7tz 1/1     Running   0           115s
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl get deployments
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
manufacturing-app 2/2       2             2           3h41m
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
kubernetes          ClusterIP   10.96.0.1     <none>       443/TCP          3h48m
manufacturing-service NodePort    10.109.135.130 <none>       5000:30007/TCP  3h41m
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl delete pod manufacturing-app-7f498dd7cd-7l54z
pod "manufacturing-app-7f498dd7cd-7l54z" deleted from default namespace
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
manufacturing-app-7f498dd7cd-7l54z 1/1     Terminating 0           3h9m
manufacturing-app-7f498dd7cd-8h7tz 1/1     Running      0           2m44s
manufacturing-app-7f498dd7cd-jxg76 0/1     Running      0           10s
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11>
```

The Kubernetes cluster shows two running pods, each running the manufacturing application container. This confirms replica configuration and successful scheduling of pods.

3.6 NodePort Service Verification

```
PS C:\Users\aadit\OneDrive\Desktop> cd "Group 06 D11"
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> dir

Directory: C:\Users\aadit\OneDrive\Desktop\Group 06 D11

Mode                LastWriteTime         Length Name
----                -
da---l            16-02-2026      10:06             app
da---l            18-02-2026      10:09             venv
-a---l            18-02-2026      10:09         812 deployment.yaml
-a---l            16-02-2026      10:08         202 Dockerfile
-a---l            16-02-2026      10:00             0 Readme.md
-a---l            16-02-2026      10:09         210 service.yaml

PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl apply -f deployment.yaml
deployment.apps/manufacturing-app created
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl apply -f service.yaml
service/manufacturing-service created
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
manufacturing-app-6df84b6b5-7vxx7   1/1     Running   0           30s
manufacturing-app-6df84b6b5-bmnn9   1/1     Running   0           30s
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl get svc
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes          ClusterIP   10.96.0.1    <none>        443/TCP          7m17s
manufacturing-service NodePort    10.109.135.130 <none>        5000:30007/TCP   50s
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> docker build -t aaditibhale/manufacturing-app:v2 .
[+] Building 14.8s (11/11) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.2s
=> => transferring dockerfile: 241B                                0.1s
```

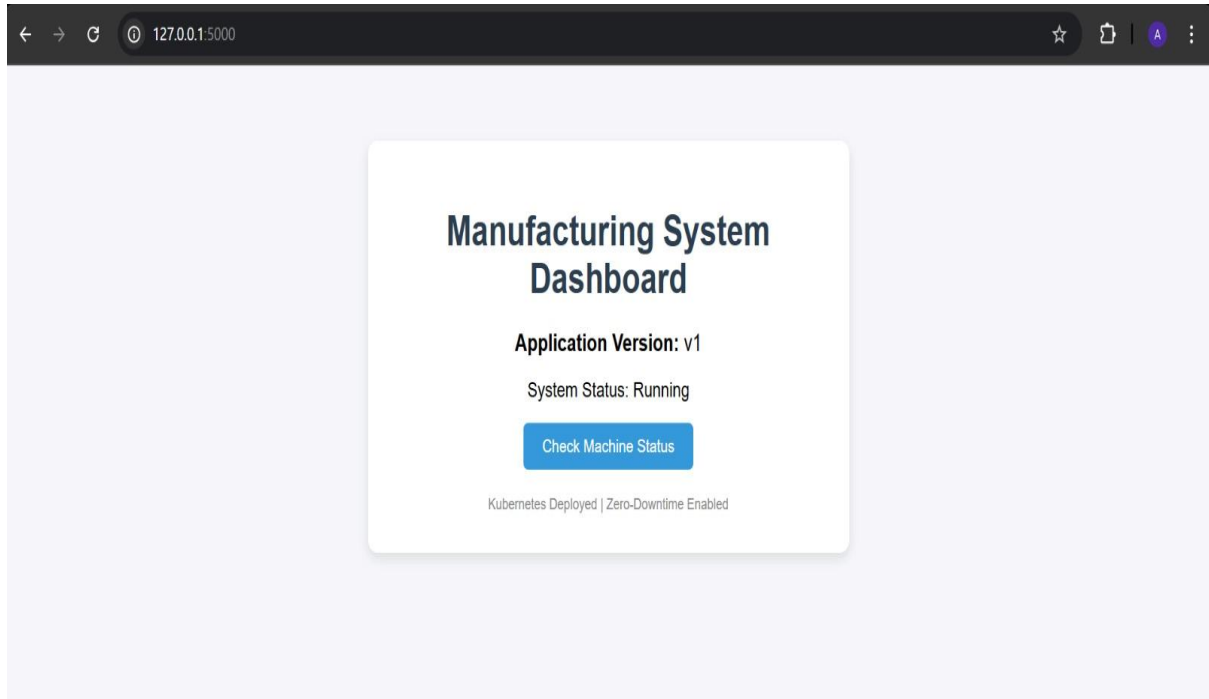
The service was exposed on:

Port 30007 (NodePort)

This allows external users to access the application via:

<http://localhost:30007>

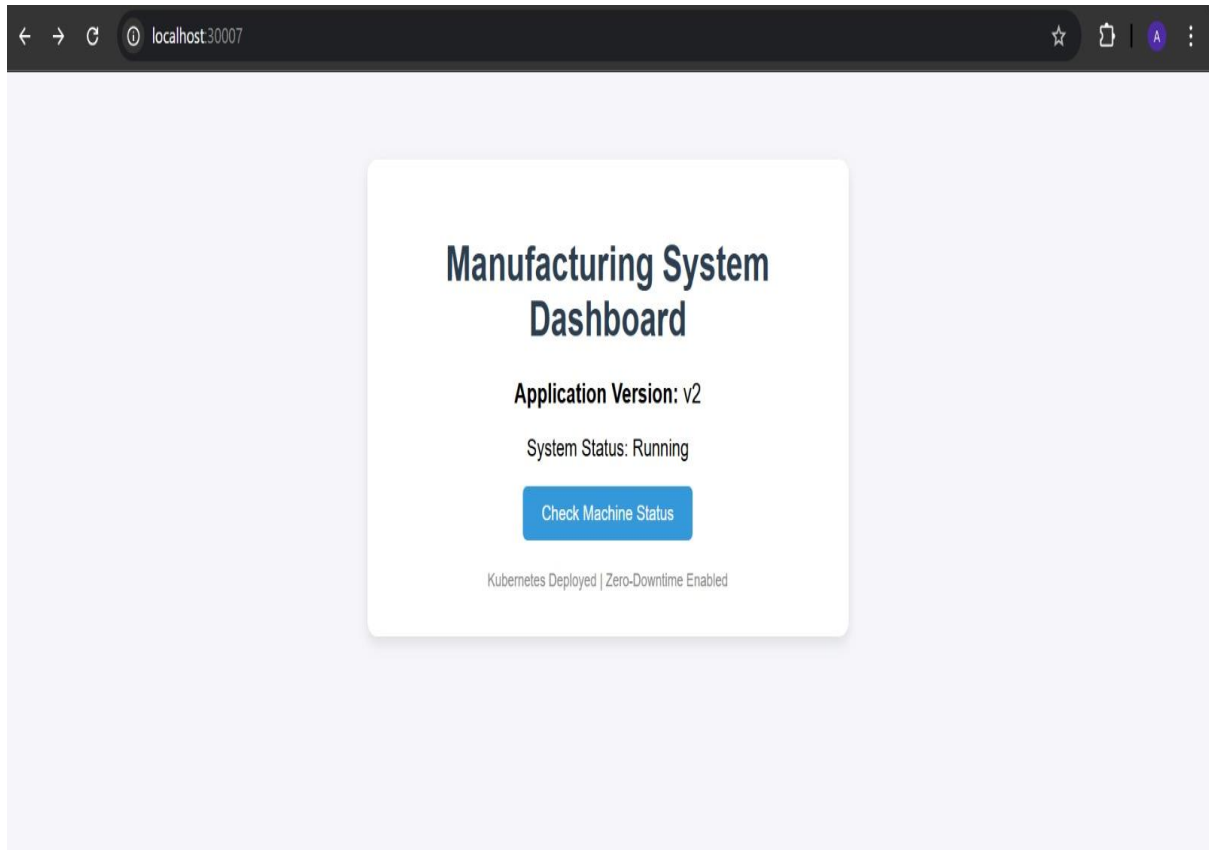
3.7 Application UI – Version v1



The application version v1 was successfully deployed and accessible via NodePort. The dashboard confirms:

- Application version
- System status
- Kubernetes deployment indicator

3.8 Application UI – Version v2 (Rolling Update)



After updating the deployment to version v2, Kubernetes performed a rolling update. Pods were replaced gradually without downtime.

The application version changed from v1 to v2 seamlessly.

This confirms zero-downtime deployment.

3.9 Pod Self-Healing Demonstration

```
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
manufacturing-app-7f498dd7cd-7l54z  1/1     Running   0           3h7m
manufacturing-app-7f498dd7cd-8h7tz  1/1     Running   0           33s
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
manufacturing-app-7f498dd7cd-7l54z  1/1     Running   0           3h8m
manufacturing-app-7f498dd7cd-8h7tz  1/1     Running   0           115s
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl get deployments
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
manufacturing-app  2/2      2             2           3h41m
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl get svc
NAME    TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes  ClusterIP   10.96.0.1    <none>        443/TCP          3h48m
manufacturing-service  NodePort    10.109.135.130  <none>        5000:30007/TCP   3h41m
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl delete pod manufacturing-app-7f498dd7cd-7l54z
pod "manufacturing-app-7f498dd7cd-7l54z" deleted from default namespace
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
manufacturing-app-7f498dd7cd-7l54z  1/1     Terminating   0           3h9m
manufacturing-app-7f498dd7cd-8h7tz  1/1     Running        0           2m44s
manufacturing-app-7f498dd7cd-jxg76  0/1     Running        0           10s
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11>
```

Sort by

Newest

Filter tags

TAG

v2

Last pushed about 3 hours by aaditibhale

docker pull aaditibhale/manufacturing-app:v2

Digest	OS/ARCH	Last pull	Compressed size
85a42de24c2a	linux/amd64	less than 1 day	46.57 MB

TAG

v1

Last pushed 4 days by aaditibhale

docker pull aaditibhale/manufacturing-app:v1

Digest	OS/ARCH	Last pull	Compressed size
5d3da9687255	linux/amd64	less than 1 day	46.57 MB

One pod was manually deleted using:

`kubectl delete pod <pod-name>`

Kubernetes automatically recreated a new pod to maintain the desired replica count.

This demonstrates self-healing capability of Kubernetes.

4. Reports / Dashboards / Models

The deployment strategy used in this project is Rolling Update Strategy.

Rolling Update works as follows:

1. New version of container image is pushed to Docker Hub.
2. Deployment YAML file is updated with new image tag.
3. Kubernetes gradually creates new pods.
4. Old pods are terminated only after new pods become ready.
5. Traffic is shifted automatically without downtime.

Advantages:

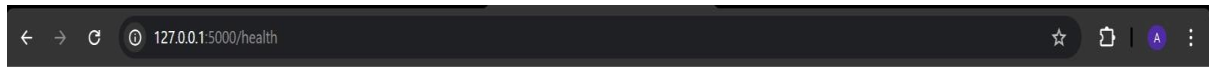
- Zero downtime
- Controlled release
- Automatic rollback support
- Smooth version transition

The Manufacturing System Dashboard provides:

- Version visibility
- Operational status
- Health endpoint validation
- Deployment confirmation

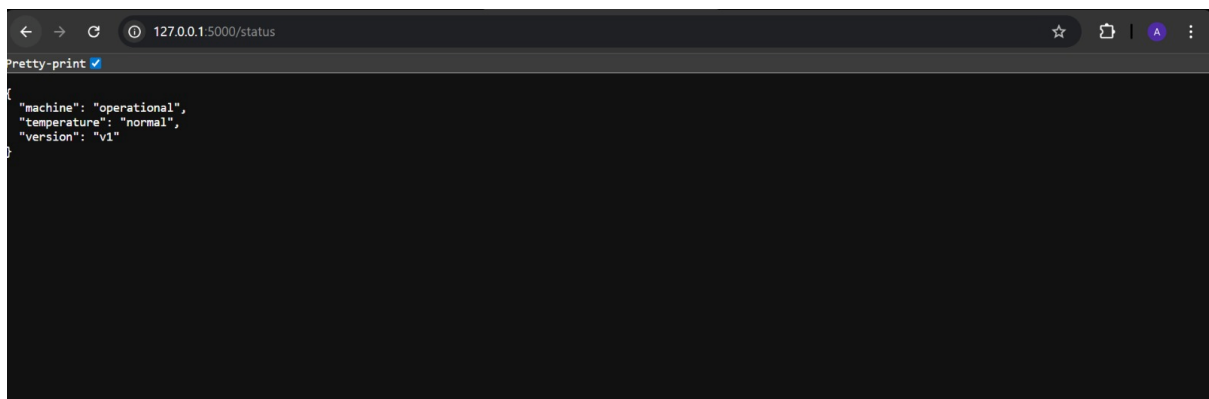
Health Endpoint:

<http://localhost:30007/health>



Status Endpoint:

<http://localhost:30007/status>



These endpoints allow integration with monitoring systems.

4.1. System Testing & Validation

To verify system reliability and functionality, several tests were performed.

Pod health was checked using `kubectl get pods`, confirming that all pods were in the *Running* state. Service exposure was verified using `kubectl get svc`, which confirmed that the NodePort service was successfully created and accessible.

The application was accessed via <http://localhost:30007>, and the dashboard loaded correctly, confirming successful deployment.

During the version upgrade test, the application was updated from v1 to v2 using a rolling update strategy. The version change occurred without any downtime, validating zero-downtime deployment capability.

In the self-healing test, a running pod was manually deleted using `kubectl delete pod`. Kubernetes automatically recreated a new pod to maintain the desired replica count, demonstrating automatic recovery.

4.2. Performance & Reliability

The system demonstrates high availability through multiple replicas and automatic failure recovery. Its stateless container architecture allows smooth scaling and efficient resource utilization. The rolling update strategy ensures controlled deployments without service interruption.

Although deployed locally, the architecture follows production-ready design principles.

4.3. DevOps Principles Applied

The project applies key DevOps concepts, including Infrastructure as Code through YAML configuration, containerization using Docker, version control via image tagging, automated orchestration using Kubernetes, zero-downtime deployment, and self-healing infrastructure.

These practices align with modern enterprise deployment standards.

4.4. Security Considerations

The system uses container isolation, controlled image versioning, and managed service exposure. While basic security measures are implemented, future enhancements may include RBAC, secrets management, HTTPS configuration, and network policies.

4.5. Comparative Analysis

Compared to traditional deployments, the containerized approach significantly reduces downtime, enables automatic scalability, supports self-healing recovery, improves environment consistency, and simplifies version control.

This clearly highlights the advantages of Kubernetes-based deployment.

5. Key Outcomes

The successful deployment of the Manufacturing Application using Docker and Kubernetes resulted in significant technical and operational achievements. The project effectively demonstrated the practical implementation of containerization and orchestration concepts in a real-world deployment scenario.

1. Application Containerization

The Manufacturing Application was successfully containerized using Docker. All dependencies, configurations, and source code were packaged into a lightweight image, ensuring:

- Environment consistency
- Elimination of dependency conflicts
- Easy portability
- Simplified deployment

Version tagging (v1 and v2) enabled structured release management.

2. Kubernetes Orchestration

The application was deployed using Kubernetes, which managed:

- Pod scheduling
- Replica management
- Service exposure
- Automatic restarts

This demonstrated efficient lifecycle management of containerized applications.

3. Zero-Downtime Rolling Updates

The upgrade from version v1 to v2 was performed using rolling updates without any service interruption. This ensured:

- Continuous availability
- Controlled deployment
- Smooth traffic transition

Such deployment strategies are critical in production environments.

4. High Availability

Multiple replicas were configured to ensure that even if one pod fails, others continue serving requests. This improves system reliability and reduces service disruption.

5. Self-Healing Capability

When a pod was manually deleted, Kubernetes automatically recreated it to maintain the desired state. This confirms automated recovery and infrastructure resilience.

6. Version Management

Maintaining multiple image versions in Docker Hub allowed controlled release management, version tracking, and potential rollback capability.

7. External Access via NodePort

The application was exposed externally using NodePort, enabling browser-based access and demonstrating practical service routing in Kubernetes.

8. Deployment Efficiency

Compared to traditional deployments, this approach reduced configuration complexity, setup time, and manual intervention, resulting in a consistent and automated deployment process.

9. Practical DevOps Learning

The project provided hands-on experience in:

- Docker image creation
- Image tagging and pushing
- Writing Kubernetes YAML files
- Managing deployments and services
- Performing rolling updates

This strengthened understanding of modern DevOps workflows.

10. Scalability and Reliability Foundation

The architecture supports future enhancements such as autoscaling and cloud deployment. Through replication, rolling updates, and self-healing, the system achieves improved reliability and operational stability.

6. Practical Industry Relevance

The technologies used in this project are actively used in:

- Cloud-native startups
- Enterprise DevOps teams
- SaaS platforms
- Manufacturing automation systems
- Microservices architecture deployments

This project simulates real-world DevOps workflow.

7. Limitations

- No database integration
- No CI/CD automation
- Local cluster only
- Basic monitoring
- No autoscaling

8. Conclusion

- The project successfully deployed a containerized Manufacturing Application using Docker and Kubernetes.
- The system ensures high availability and scalability through multiple replicas.
- Zero-downtime deployment was achieved using the rolling update strategy.
- Kubernetes automatically recreates failed pods, demonstrating self-healing capability.
- Docker enabled consistent packaging of the application with all dependencies.
- Kubernetes handled orchestration, deployment management, and service exposure.
- The project simulates a production-grade deployment environment.
- It reflects real-world DevOps practices used in modern cloud-native systems.
- Practical knowledge was gained in containerization, orchestration, version control, and deployment automation.
- The implementation lays a strong foundation for future cloud-native and enterprise-level deployments

9.Future Scope & Enhancements

The current implementation provides a strong foundation for containerized application deployment. However, the system can be further enhanced to achieve enterprise-level scalability, security, and automation. The following improvements can be implemented in future phases:

1. Scalability Enhancements

- **Horizontal Pod Autoscaler (HPA):**
Automatically scale pods based on CPU or memory utilization to handle varying traffic loads.
- **Cloud-Based Deployment (AWS EKS / Azure AKS):**
Deploy the application on managed Kubernetes services for production-grade scalability and availability.

2. Advanced Traffic Management

- **Ingress Controller Integration:**
Implement load balancing and domain-based routing for better traffic control.
- **Blue-Green Deployment Strategy:**
Introduce advanced deployment strategies to enable safer releases and faster rollback capabilities.

3. DevOps Automation

- **CI/CD Pipeline Integration (GitHub Actions / Jenkins):**
Automate build, test, and deployment processes to ensure continuous integration and delivery.
- **Automated Image Scanning & Validation:**
Enhance deployment reliability by integrating security checks within the pipeline.

4. Monitoring & Observability

- **Prometheus & Grafana Integration:**
Enable real-time monitoring, performance visualization, and alerting mechanisms.
- **Centralized Logging (ELK Stack):**
Implement Elasticsearch, Logstash, and Kibana for structured log collection and analysis.

5. Security Improvements

- **Role-Based Access Control (RBAC):**
Implement fine-grained access policies within the Kubernetes cluster.
- **Secrets Management:**
Securely manage sensitive data such as credentials and API keys.
- **HTTPS & TLS Configuration:**
Encrypt network communication for secure external access.

6. Data & Storage Integration

- **Persistent Storage Integration:**
Add Kubernetes Persistent Volumes for stateful workloads.
- **Database Integration:**
Connect the application to relational or NoSQL databases for real-world manufacturing data management.